

임채현

BACKEND DEVELOPER

AI를 어디까지 쓸지 판단하는 개발자

LLM·알고리즘 경계 설계

인증·신뢰 경계 설계

검색·파이프라인 최적화

chaehyun010104@gmail.com · github.com/Chaehyunli · ch010104.tistory.com/

ABOUT

- 문제를 먼저 정의하고, LLM에 맡길 부분과 코드로 확정할 부분의 경계를 나눈 뒤 구현한다.
- 판단마다 기각한 대안과 트레이드오프를 기록으로 남기고, 성능·프롬프트 변경은 골든셋·회귀 fixture로 검증한다.
- 팀 작업에서는 트러블슈팅과 응답 계약을 먼저 문서로 고정해, 서로의 구현을 기다리지 않고 병렬로 진행하고 필요하면 이어받을 수 있게 한다.
- 다음으로는 백엔드 파이프라인에 AI Agent를 붙이는 작업 — 요청 분류·구간별 검증·호출량 제어까지 포함한 설계 — 를 더 깊게 다루고 싶다.

SKILLS

LANGUAGES	Java · Python · TypeScript · SQL
BACKEND	Spring Boot · Spring Security · Spring WebFlux / R2DBC · FastAPI · JPA / SQLModel · WebSocket·STOMP · SSE
DATA · SEARCH	PostgreSQL · MySQL · Redis · Elasticsearch / OpenSearch · pgvector · Alembic / Flyway
AI 파이프라인	PydanticAI · DeepEval 골든셋·회귀 fixture · RAG · KeyBERT · XGBoost
INFRA	Docker Compose · Vercel · Render · Git

CAREER

2026.01 ~ 2026.02

프람트테크놀로지 · 개발팀 인턴 — 레거시 → MSA 전환 품질 검증

약 20만 LOC 전략물자 관리 시스템을 4개 서버 모듈(portal·permission·judgement·destination)

MSA 구조로 전환하는 프로젝트에서, 레거시 동작을 검증 기준으로 복원하는 일을 맡았다.

- 문서화되지 않은 레거시 화면·조건 분기·산출물을 역추적해 상태별 테스트 기준을 세우고, 신청 시나리오와 상태 전이를 분리해 약 8,000개 단위·통합 테스트 케이스를 설계·수행했다.
- 결함을 현상이 아니라 레거시와의 차이·재현 절차·발생 모듈 관점으로 정리해 전달했고, 필수 첨부 조건으로 막힌 로컬 제출 테스트는 업로드 경로를 조정해 검증 공백을 해소했다.
- "상세 화면에서 목록으로 돌아가도 검색 조건 유지" 요구를 받아, 각 MSA 모듈이 검색 필터를 query 파라미터와 response body에 혼용하는 것을 발견하고 query 파라미터로 통일하는 방안을 개발팀에 제안했다.

Spring Boot

ThymeLeaf

MSA

EDUCATION & TRAINING

2021.03 ~ 2026.08 — 명지대학교 컴퓨터공학부 컴퓨터공학과 졸업

2025.07 ~ 2025.08 — SK AI Dream Camp 중급

2026 ~ 현재 — SKALA 4기 — Agile/Scrum·MSA 팀 실습(SM), 응답 계약 기반 병렬 개발

자격 — 정보처리기사 · SQLD

2026.08 ~

개인 프로젝트

배포 운영 중

맡은 역할

기획·설계

구현

배포·운영

FastAPI

SQLModel

PostgreSQL

Redis

Alembic

Vue 3

TypeScript

Pinia

Tailwind CSS

Web Crypto API

DeepEval

Vercel

Render

배포: moduyaksok.vercel.app · GitHub: github.com/Chaehyunli/moduyaksok · API 문서: moduyaksok.onrender.com/docs

🤝 모두약속

목적·시간·지역·예산·선호를 입력하면 실제 장소 검색 결과로 약속 일정을 만들어주는 AI 서비스

배경

왜 만들었나



실제 배포된 랜딩 페이지 - 낙서하듯 적으면 일정이 나온다

반복되는 장소·동선 조율

약속마다 매번 새로 고르고 맞춰야 하는 번거로움

일반적인 AI 추천의 한계

존재하지 않는 장소를 생성, 실제 이동거리 미고려

실제 검색 결과 기반 설계

검증된 장소 안에서만 일정 구성 + 실제 경로로 동선 검증

설계

어떻게 동작하나



일정 후보 3개 제시

네이버 지역검색 결과만 후보로 사용

LLM이 존재하지 않는 장소를 만드는 걸 구조적으로 차단

관점 3개 병렬 생성

하나의 정답이 아니라 서로 다른 기준의 선택지 제공

Step 4를 사용자 선택 후로 미룸

선택된 후보에만 호출해 API 비용 절약

Vue 3 + Web Crypto API

브라우저 내장 암호화로 키 암호화를 클라이언트에서 처리

판단 01

저장된 사용자 키를 서버가 단독으로 복호화하지 못하게 했다

SITUATION & TASK · 마주한 문제와 고민

LLM 호출 비용을 운영자가 부담하지 않도록 사용자가 자신의 API 키를 등록하는 BYOK 방식을 선택했다. 처음에는 서버의 마스터키로 API 키를 암호화해 DB에 저장했지만, 서버와 DB가 함께 침해되거나 운영자가 마스터키에 접근하면 모든 사용자의 키를 복호화할 수 있었다. 다만 provider 호출과 사용량 제어를 서버에서 수행하려면 요청 처리 순간에는 서버가 평문 키를 받아야 했기 때문에, 저장 시점과 호출 시점의 신뢰 경계를 분리해야 했다.

- 클라이언트가 provider를 직접 호출 — 서버를 신뢰 경계에서 제외할 수 있지만 CORS·provider별 SDK·사용량 제어를 프론트에서 처리해야 함
- 키를 저장하지 않고 호출마다 입력 — 저장 위험은 없어지지만 사용성이 떨어지고, 서버 경우 호출이라면 처리 순간 평문을 보는 점은 동일함
- 서버 마스터키로 암호화 — 구현은 단순하지만 서버 침해 시 저장된 모든 키를 복호화할 수 있음

ACTION · 나의 판단과 행동

저장용 암호화와 호출용 평문 처리를 분리

저장된 키는 서버가 단독으로 복호화할 수 없게 하고, provider 호출 순간의 평문 처리는 서버가 맡는 구조로 신뢰 경계를 명확히 나눴다.

패스프레이즈로 유도한 키, 브라우저에서 암호화

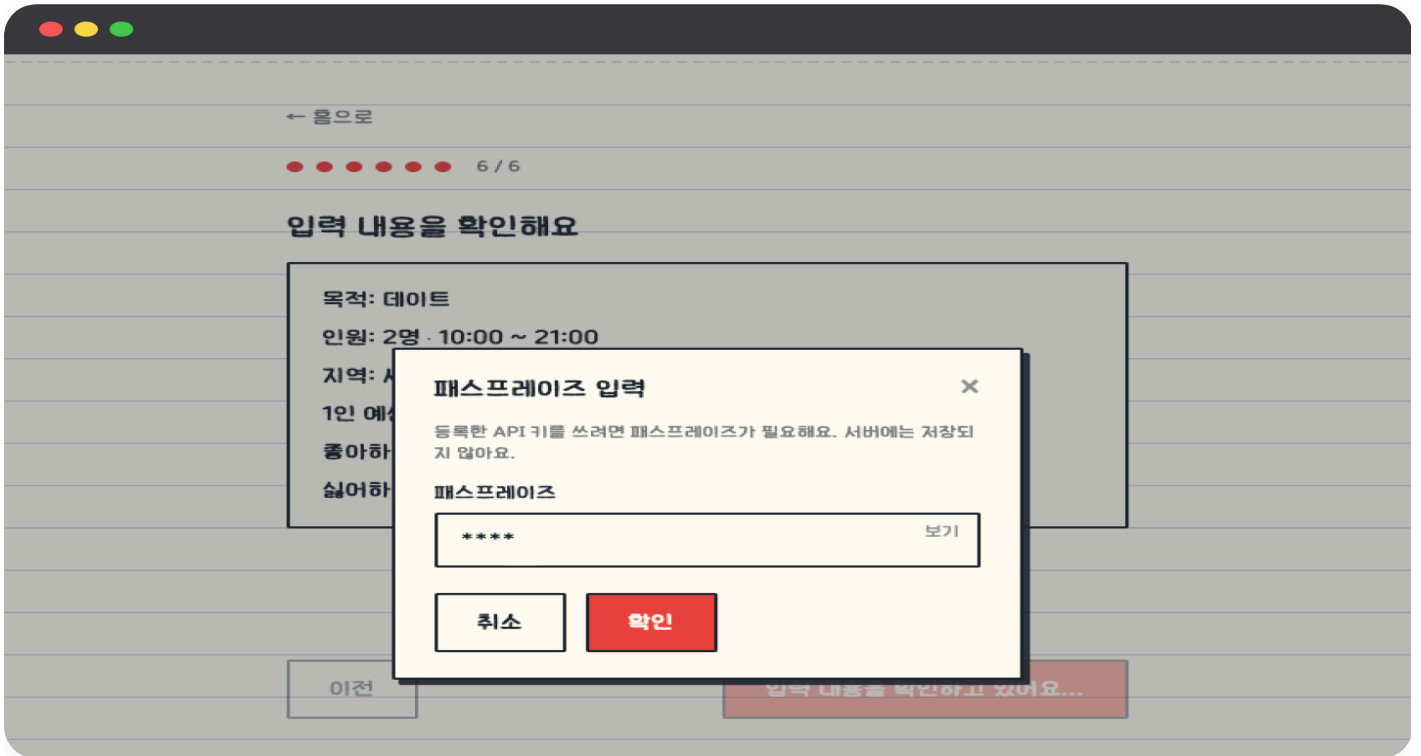
PBKDF2-HMAC-SHA256 60만 회로 키를 유도하고 AES-GCM 256bit으로 브라우저에서 암호화했다. 서버에는 암호문·salt·iv·KDF 반복 횟수만 저장하고 패스프레이즈와 유도 키는 전달하지 않았다.

Argon2 대신 crypto.subtle PBKDF2

Argon2가 더 강하지만 WASM 의존성이 붙어, 브라우저 내장 API만으로 구현 가능한 PBKDF2를 선택했다. OWASP의 PBKDF2-HMAC-SHA256 권장치인 600,000회를 적용하고 반복 횟수를 함께 저장해 이후 상향할 수 있게 했다.

평문은 HTTPS 요청 범위를 벗어나 보관하지 않음

브라우저가 복호화한 키를 provider 호출 요청 한 건의 본문으로 전달하고, 서버는 해당 호출에만 사용했다. 평문을 DB·Redis·세션에 저장하지 않고 요청 본문 로그·예외·에러 추적 도구에서도 제외했다. 유도 키는 sessionStorage에만 두어 탭을 닫으면 사라지게 했다.



패스프레이즈 입력 - 서버에는 저장되지 않는다

RESULT · 결과

서버 마스터키로 모든 사용자 키를 복호화할 수 있던 Fernet 구조에서, 저장된 암호문을 서버가 단독으로 복호화할 수 없는 구조로 전환했다. DB나 백업만 유출돼서는 API 키 평문이 바로 노출되지 않는다. 다만 provider 호출 순간에는 서버가 평문을 처리하므로 런타임 서버는 여전히 신뢰 경계에 포함되며, 이는 CORS·사용량 제어·프록시 정책을 서버에서 유지하기 위해 수용한 트레이드오프다.

판단 02

LLM이 할 일과 코드가 할 일을 나눴다

SITUATION & TASK · 마주한 문제와 고민

프롬프트에 "같은 태그는 후보당 최대 1곳", "점심·저녁 시간대면 식사 장소를 넣어라"를 명시해도 지켜지지 않았다. 직접 써보며 "와플" 같은 강한 선호 태그에 디저트·카페만으로 하루가 채워져 식사가 빠진 일정, 여러 지역을 입력하면 멀리 떨어진 지역이 한 코스로 섞이는 일정이 반복됐다. 프롬프트를 고쳐도 빈도만 줄 뿐 사라지지 않았다.

- 시간 겹침·예산·이동거리·식사 슬롯·태그 중복은 좌표와 시각만으로 코드가 확정 가능 — 프롬프트로 부탁할 대상이 아님
- "해산물 태그와 이자카야 카테고리가 의미적으로 겹치는가" 같은 판단만 규칙으로 못 잡음 — 이것만 LLM에 남김

ACTION · 나의 판단과 행동

계산 가능한 조건은 코드로 분리

시간 겹침·예산·이동거리·식사 슬롯·태그 중복은 좌표와 시각만 있으면 코드가 확정한다. 프롬프트로 부탁할 대상이 아니라고 보고 Step 3 앞단에 규칙 하드 필터를 설계해 넣었다.

규칙으로 못 잡는 판단만 LLM에

"해산물 태그와 이자카야가 의미적으로 겹치는가" 같은 판단만 LLM에 남기고, 규칙을 통과한 후보에만 1회 호출하도록 구성했다.

후보 생성 알고리즘 버전을 따로 구현

순차 생성하면 지역 최적점에 갇혀, beam width 80으로 완성 조합을 계획당 12개씩 만든 뒤 세 후보를 한 세트로 공동 평가한다(장소·카테고리 Jaccard, 태그 충족).

경계를 옮길 때마다 수치로 검증

LLM이 남은 Step1·Step3는 골든셋 11·4케이스(GEval 임계값 0.70, judge 고정)로, 코드가 맞은 하드조건은 결정론적 회귀 fixture로 본다. 프롬프트·모델을 바꾸면 같은 세트로 다시 돌린다.



조건 입력 — 겹치는 번호는 생성 전에 짚어준다

RESULT · 결과

하드 조건은 완성 후 검사가 아니라 beam search가 후보를 넓히는 도중부터 불변조건으로 강제된다 — 결정론적 회귀 fixture(합성 데이터)에서 세 후보 모두 위반 0건이 재현된다. Step1·Step3 골든셋은 GEval 0.70 임계값을 통과한다.

판단 03

외부 API 호출량을 제어하되 서비스는 멈추지 않게

SITUATION & TASK · 마주한 문제와 고민

네이버 지역검색은 초당 10건·일 25,000건, ODsay는 일 1,000건 한도가 있다. 장소 풀을 모으려면 카테고리 16종 × 태그 만큼 팬아웃해야 해서, 한 번의 일정 생성이 네이버를 21회 호출한다(경기 수원 조건 실측, 고유 장소 92곳).

- 재시도(backoff)만으로는 호출 폭주 자체를 못 막음 — 속도를 사전에 제한해야 함
- 순수 FIFO 토큰버킷은 한 생성 요청이 큐를 독점해 다른 사용자를 굶김
- 리미터를 in-memory로 두면 멀티 워커에서 각자 한도를 세 전역 상한이 깨짐

ACTION · 나의 판단과 행동

초당은 세션 단위 라운드로빈, 일일은 Redis

초당 상한은 세션 단위 라운드로빈 토큰버킷 — 세션 N개면 세션당 처리량이 자연히 rate/N 로 수렴한다.
일일 한도는 Redis 카운터 + 자정 TTL로 여러 워커에서도 전역 집계기 하나로 유지된다.

부족하면 막지 않고 줄인다

예산이 모자라면 요청을 차단하는 대신 확보 가능한 만큼만 검색하고, 카테고리 쿼리를 우선 생존시켜 결과 품질을 지킨다.

부를 필요 없는 호출은 생략

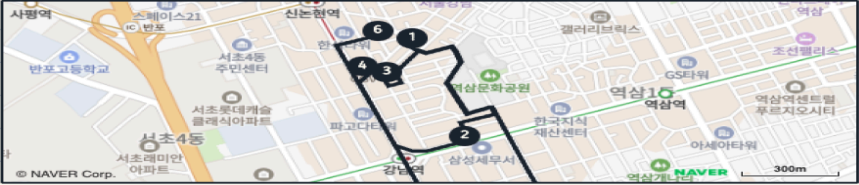
ODsay는 700m 이내 도보 구간을 사전에 걸러 호출 자체를 건너뛰는다.

실내에서 움직이는 가성비 강남 코스

참치와 음식, 카페가 섞여 있어서 와를 취향은 따로 챙기지 않더라도 갈아타지 않고 실내 위주로 움직이기 편해요. 예산 안에서 선택지를 넓게 가져가기가 좋아요.

! 확인해주세요

예정보다 약 6분 더 걸릴 수 있어요 주말에는 사람이 몰릴 수 있으니 한산한 시간을 원하시면 조금 일찍 움직이거나 명일을 고려해 보세요. 실제 이동시간을 반영하면 마지막 활동이 21:09에 끝나 희망 시간(21:00)을 넘길 수 있습니다.



1 에이비카페
카페, 디저트 > 카페 10:00-11:30
1만 5,000~12,000원
영업시간은 자동으로 확인이 안 돼요 **지도에서 직접 확인**

이 장소 배기

자차 4분

○ 도보 10분
● **자차 4분 · 108원 · 자동차(실시간 바뀜)**

지도·경로 상세 - 700m 이내 구간은 호출 없이 처리한다

RESULT · 결과

하이브리드 전환으로 생성 1건당 네이버 검색이 36회에서 21회로 줄었고, 라운드로빈 덕에 세션이 몰려도 서로 굶기지 않는다. 초당 리미터가 아직 in-process라 멀티 워커 스케일 시 전역 상한이 어긋나는 건 다음 과제로 남겼다.

공개 배포 후 본인·지인이 사용 중. 일정 1건이 네이버 지역검색 21회로 팬아웃되는 구조라 초기 생성은 로컬 5회 기준 p50 16초 / p95 ~20초이며, Step 4를 사용자 선택 이후로 미뤄 호출 비용을 줄였다. 후보는 실제 검색 결과 안에서만 구성한다.
<https://moduyaksok.vercel.app>

2026.03 ~ 2026.08

4인 팀

앱 배포 완료

맡은 역할

AI 파이프라인 설계 단독

AI 서버 구현 80%

구간별 DEEPEVAL 검증

백엔드 API 설계

일정 도메인 구현

R2DBC 비동기 전환

React Native

Expo

Spring WebFlux

FastAPI

PydanticAI

PostgreSQL

Redis

SSE

Docker Compose

GitHub: github.com/orgs/Masil2026/repositories · 담당 PR: github.com/search?q=org%3AMasil2026+author%3AChaehyunli+is%3Apr&type=pullrequests



Masil

대화로 일정을 짜고 예약까지 이어지는 AI agent 여행 플래너

배경

왜 만들었나

기존 ML 기반 여행 추천의 한계

학습된 여행지·일정 범위 안에서만 결과 생성

실시간 정보 미반영

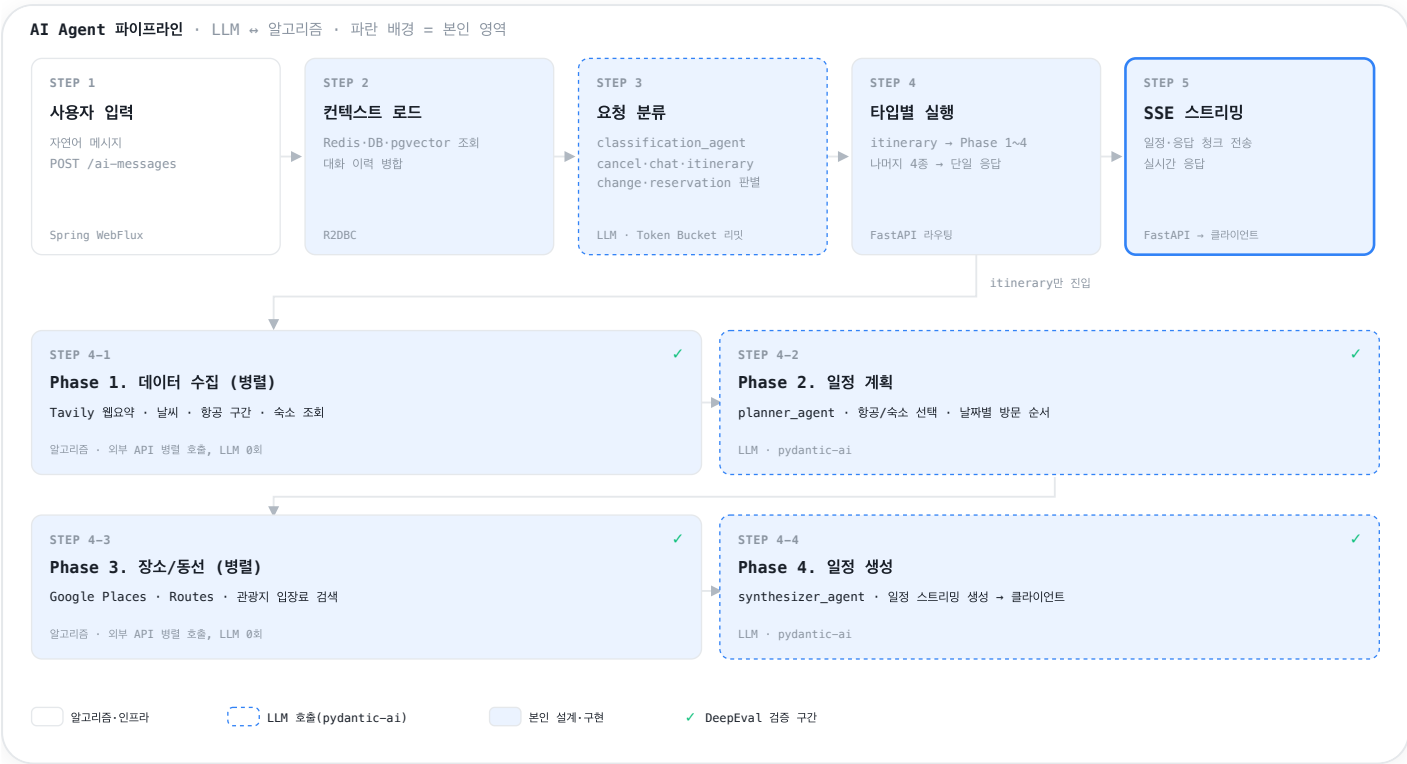
학습 데이터에 없는 지역·실시간 변경 정보 반영 불가

AI agent 기반 재설계

실시간 정보 직접 조회해 일정 구성 + 예약까지 수행

설계

어떻게 동작하나



AI 일정·예약 후보 제안

- Spring WebFlux**

AI 요청은 응답까지 오래 걸린다 — 요청 단위로 스레드를 점유하지 않기 위해
- Pydantic AI**

단계마다 출력 형태가 흔들린다 — 각 step 입출력을 스키마로 고정하기 위해

구간별 DeepEval 검증

LLM 파이프라인은 틀려도 어느 단계인지 모른다 — step마다 골든셋 테스트로 구간 특정

판단 01

되돌릴 수 없는 동작은 파이프라인에 들어가기 전에 확정했다

SITUATION & TASK · 마주한 문제와 고민

요청이 आए할 때 파이프라인이 그대로 진행되면 엉뚱한 동작이 실행됐다. "그거 취소해줘" 한마디가 일정을 새로 만들라는 건지, 예약을 취소하라는 건지 구분되지 않은 채 흘러가면 되돌릴 수 없는 예약·취소가 잘못 실행된다.

- 분류를 파이프라인 안에서 하면 이미 실행이 시작된 뒤라 되돌리기 어렵다 — 진입 전에 판단해야 한다
- 되돌릴 수 없는 동작(예약·취소)은 확정 전에 한 번 더 물어야 한다

ACTION · 나의 판단과 행동

5종 분류를 파이프라인 진입 전에 설계해 넣었다

classification_agent가 대화 요약·선호·현재 일정·예약 정보를 함께 보고 chat-itinerary-reservation-change-cancel 중 하나로 타입을 확정한다.

미확정이면 실행하지 않고 확인 단계로 분기

예약 취소는 대상을 명확히 안 골랐으면 후보를 먼저 제시하고 확인받도록 흐름을 바꿨다.

각 step에 DeepEval 골든 데이터셋 테스트

결과가 엉뚱할 때 응답 생성이 아니라 요청 해석 단계의 문제임을 특정할 수 있게 만들었다.

RESULT · 결과

모호한 취소·수정 요청이 후보 확인 없이 실행되는 경우가 사라졌다. 파이프라인을 통째로 고치지 않고 구간 단위로 수정할 수 있어, 요청 해석이 틀렸을 때 그 step만 손본다.



예약 상태 — 잘못 취소되면 되돌릴 수 없는 화면

판단 02

WebFlux 위에서 DB 접근만 블로킹으로 남아있던 문제를 없앴다

SITUATION & TASK · 마주한 문제와 고민

WebFlux로 비동기 요청 처리를 구성했는데 데이터 접근 계층이 블로킹 JPA라, 그 구간에서 비동기 흐름이 끊기고 AI 응답을 기다리는 동안 요청 스레드가 다시 묶였다.

- WebFlux + 블로킹 JPA는 구조적 모순 — 비동기의 이점이 DB 구간에서 전부 사라진다
- R2DBC는 라이브러리 교체가 아니라 계층 전환 — 직렬화·지연 평가·Redis 접근이 함께 바뀌어야 반쪽이 안 된다

ACTION · 나의 판단과 행동

R2DBC로 데이터 접근 계층까지 완전 비동기화

요청 처리 전 구간의 논블로킹 흐름을 맞췄다.

라이브러리 교체로 끝내지 않고 주변까지 보완

Reactive Redis·JSON 변환기·테스트를 함께 정비하고, 전환 중 만난 lazy evaluation 오류를 수정해 비동기 체인 실행 시점을 안정화했다.

RESULT · 결과

요청 처리 전 구간이 논블로킹으로 이어져, AI 응답을 기다리는 동안 요청 스레드가 묶이지 않는다. 전환을 직렬화·지연 평가·테스트까지 검증하고 마무리했다.



Day별 일정 상세 - 비동기 파이프라인이 만들어 내려보내는 결과

판단 03

LLM API 호출 폭주로 대화 흐름이 끊기는 문제를 막았다

SITUATION & TASK · 마주한 문제와 고민

LLM API가 429를 반환하면 진행 중이던 대화 응답이 그대로 끊겼다. 사용자가 몰리거나 한 대화에서 agent가 여러 번 호출하면 순간 호출량이 한도를 넘겼다.

- 재시도(backoff)만으로는 이미 폭주한 호출을 늦출 뿐, 폭주 자체를 못 막는다 — 속도를 사전에 제한해야 한다
- 고정 간격 재시도는 여러 요청이 같은 타이밍에 몰려 다시 429를 부른다

ACTION · 나의 판단과 행동

Token Bucket으로 호출 속도를 사전에 완화

한도를 넘기 전에 호출 간격을 벌린다.

429 발생 시 exponential backoff + jitter 재시도

지수적으로 간격을 늘리고 무작위 지연을 섞어 재요청이 서로 겹치지 않게 했다.

순간 호출량이 한도를 넘기 전에 놀리고, 그래도 429가 나면 대화가 끊기지 않고 재개된다. 재시도 로직은 테스트로 검증했다.



당일 진행 - SSE 스트림으로 실시간 갱신되는 화면

판단 04

팀이 서로의 구현을 문서만으로 이어받을 수 있게 만들었다

SITUATION & TASK · 마주한 문제와 고민

4인이 프론트·백엔드·AI 서버로 나눠 개발하다 보니 초반에는 각자 맡은 영역 밖의 구현을 서로 잘 몰랐다. API 명세와 ERD만으로는 왜 그 기술을 골랐고 어디서 막혔는지가 공유되지 않아, 통합 시점에 인수인계 비용이 컸다.

- API 명세·ERD는 결과만 담아 트러블슈팅 맥락과 기술 선정 이유가 빠진다
- 한 사람이 자리를 비우면 그 구간이 통째로 멈추는 구조는 전시회 일정에 위험하다

ACTION · 나의 판단과 행동

트러블슈팅·기술 선정 이유를 노선에 남기는 컨벤션을 제안해 도입했다

프론트·백엔드·AI 서버별로 겪은 문제와 선택 근거를 각자 노선 페이지에 기록하게 했다. 명세가 아니라 "왜 이렇게 했는가"를 남기는 것이 목적이었다.

주 1회 회의에서 각자 페이지를 기준으로 팀 전체에 브리핑했다

정리한 내용을 매주 공유해, 자기 영역 밖의 진행 상황과 막힌 지점을 팀이 계속 따라올 수 있게 했다.

RESULT · 결과

전시회 준비 기간 중 내가 AI 서버 작업을 이어갈 수 없게 됐을 때, 팀원이 축적된 문서만으로 AI 파이프라인 최적화 마무리를 이어받아 앱 배포까지 완료했다.

2025.10 ~ 2025.12

개인 프로젝트

완성

맡은 역할

기획·설계

태그·파이프라인 구현

검색·RAG 구현

FastAPI

PostgreSQL

Elasticsearch

pgvector

Redis

MinIO

KeyBERT

React

TypeScript

Docker Compose

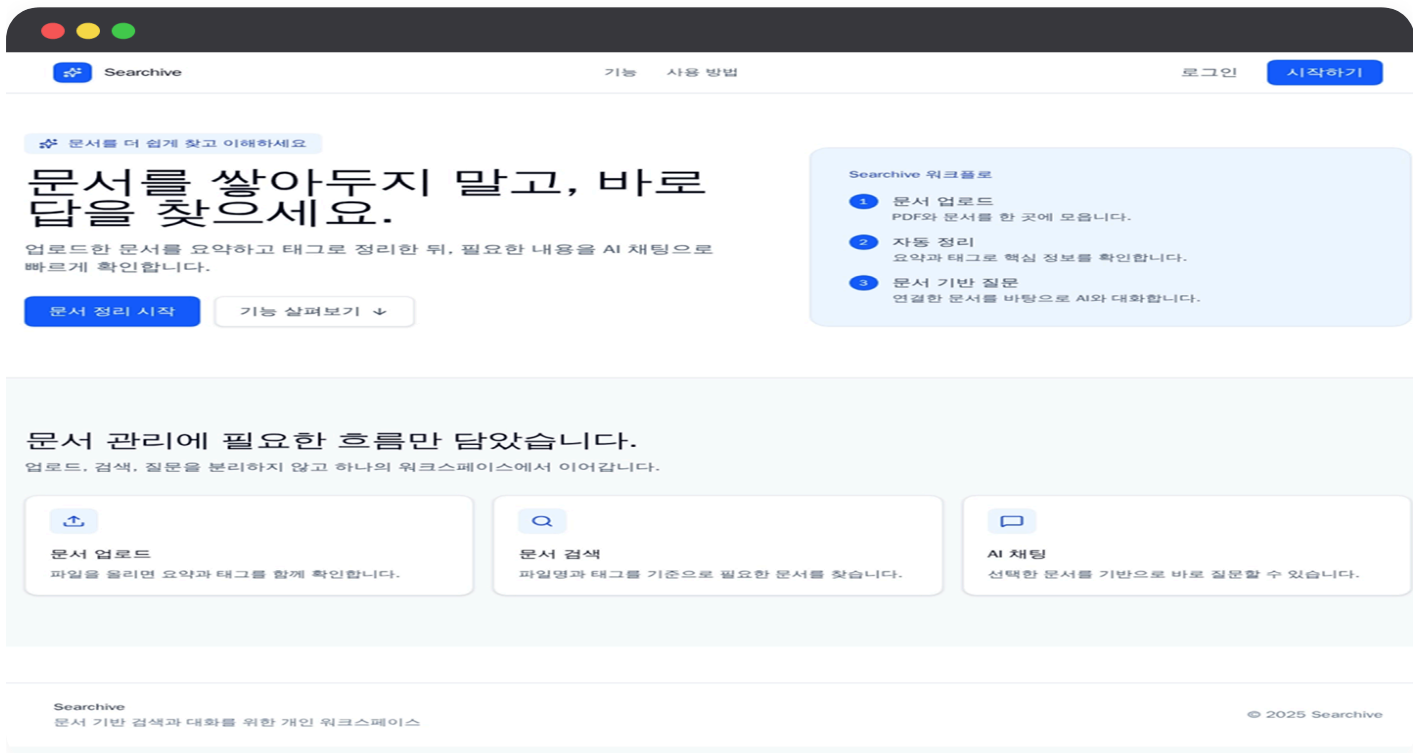
GitHub: github.com/orgs/Searchive-Project/repositories

Searchive

문서를 업로드하면 자동 태깅·검색·RAG 질의응답으로 이어지는 개인 지식 베이스

배경

왜 만들었나



랜딩 페이지 - 업로드·정리·질의응답을 하나의 흐름으로 안내

폴더에 쌓아두면 못 찾는 문서

나중에 어디에 뭐가 있는지 찾기 어려움

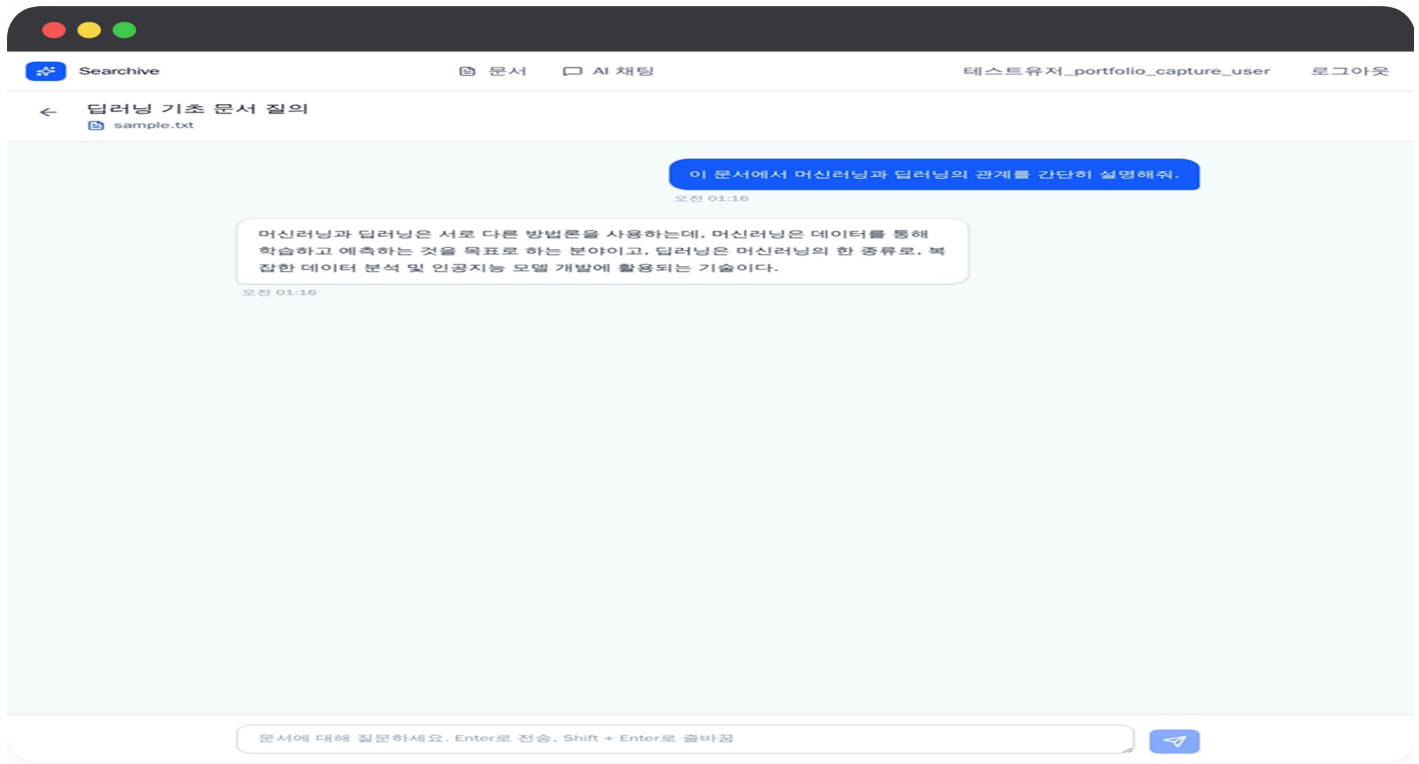
외부 서비스 의존 없이 자동 분류

자동 태깅 → 검색 → 질의응답으로 이어지는 구조 필요

업로드→태깅→검색→RAG 단일 파이프라인

사용 흐름

검색 결과를 근거로 답을 받는다



RAG 질의응답 - 검색 결과를 근거로 답변을 구성한다

문서 업로드와 색인

PDF·텍스트에서 내용을 추출해 검색 가능한 문서로 색인한다.

자동 태깅과 대표 태그 재사용

후보를 정제하고 기존 태그와 연결해 검색 필터를 일관되게 유지한다.

검색 결과를 근거로 답변 생성

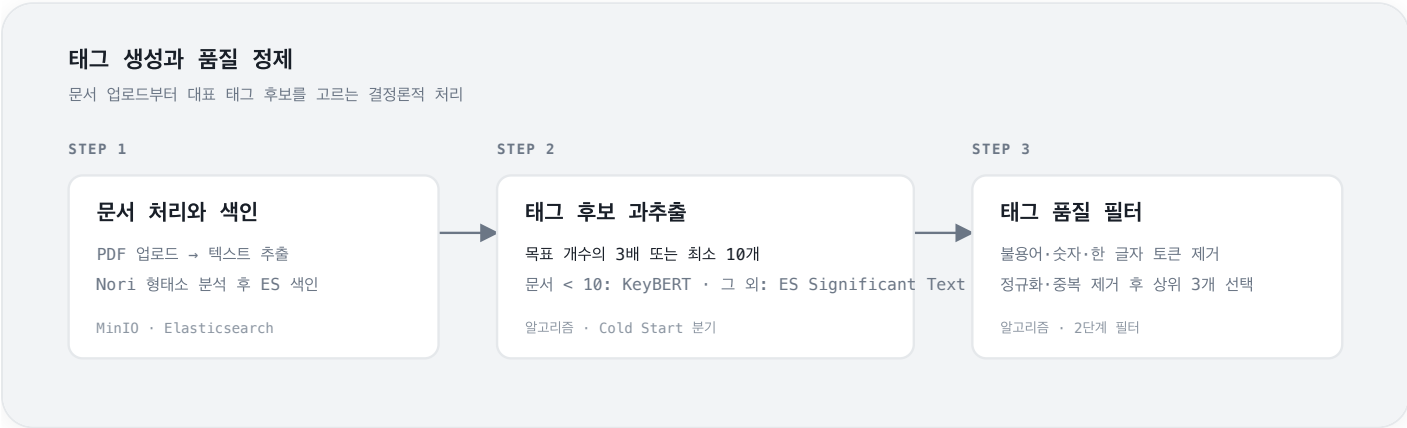
찾아낸 문서 정보를 바탕으로 RAG 응답을 구성한다.

AI가 뽑은 후보를 그대로 저장하지 않고 과추출+필터+대표 태그 수렴으로 정제했다

SITUATION & TASK · 마주한 문제와 고민

KeyBERT가 문서에서 뽑은 키워드에는 불용어·숫자·한 글자 토큰이 섞였고, 같은 의미라도 표기가 조금 다르면 별도의 태그로 저장됐다. 후보를 그대로 저장하면 검색 필터가 잡음으로 늘어나고, 시간이 지날수록 하나의 개념이 여러 태그로 흩어졌다.

- 추출 개수를 목표 태그 수만큼만 제한하면 필터링 뒤 남는 후보가 부족해, 품질 기준을 적용할 여지가 사라진다
- 문자열 완전 일치만 쓰면 표기 차이를 흡수하지 못하고, 반대로 유사도만 쓰면 다른 의미를 같은 태그로 합칠 위험이 있다
- 매번 새 태그를 생성하면 구현은 단순하지만, 태그 사전이 누적될수록 검색·필터의 일관성이 무너진다



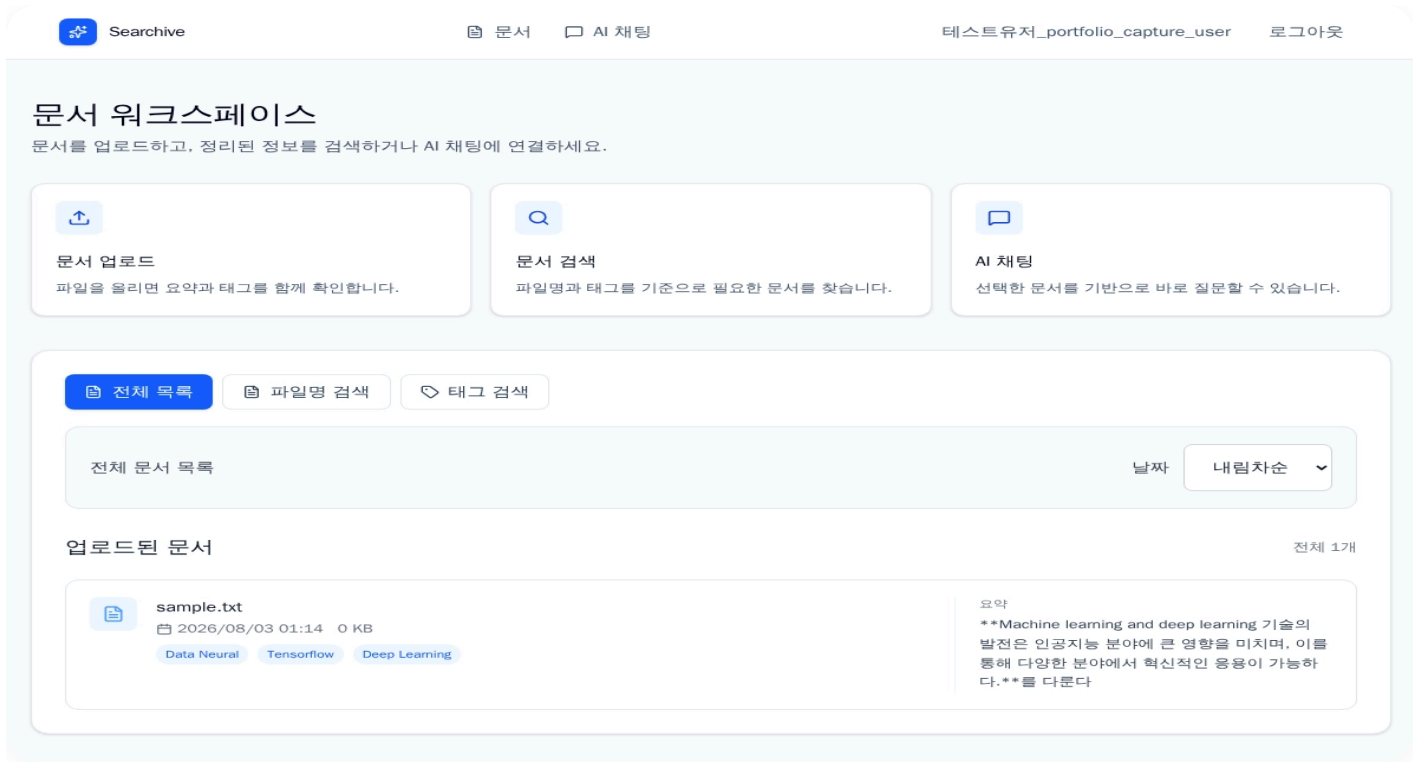
ACTION · 나의 판단과 행동

목표 개수의 3배를 먼저 뽑고 품질 기준으로 거르기

초기 후보를 넉넉히 확보한 뒤 불용어·숫자·한 글자 토큰을 제거하고 표기를 정규화했다. 중복을 건어낸 결과에서 상위 3개만 선택해, 추출 단계의 잡음이 저장 단계까지 이어지지 않게 했다.

완전 일치 다음에 벡터 유사도 0.8 이상으로 대표 태그 찾기

정규화한 이름이 기존 태그와 정확히 같으면 즉시 재사용하고, 없을 때만 벡터 유사도 0.8 이상인 대표 태그에 수렴시켰다. 두 기준을 모두 통과하지 못한 후보만 신규 태그로 만들었다.



문서 업로드 뒤 정제된 자동 태깅 결과

RESULT · 결과

태그 저장은 추출기의 출력을 그대로 쌓는 방식에서, 품질 필터와 기존 태그 재사용을 거친 뒤 새 태그를 만드는 방식으로 바뀌었다. 같은 개념의 표기 차이가 검색 필터를 분산시키는 일을 줄이고, 사용자가 화면에서 보는 태그를 더 일관되게 유지했다.

판단 02

계산과 통신이 함께 N배로 늘어나는 구조를 배치 검색으로 풀었다

SITUATION & TASK · 마주한 문제와 고민

문서에서 추출한 태그 후보마다 기존 대표 태그를 찾으려고 Elasticsearch KNN 검색을 개별 요청으로 보냈다. 후보가 늘어날수록 같은 검색 단계에서 KNN 연산과 HTTP 왕복이 함께 N배로 늘어났고, 태그 품질과 무관한 요청 대기 시간이 업로드 흐름의 병목이 됐다.

- 후보를 하나씩 순차 처리하면 각 검색 결과를 독립적으로 확인하기는 쉽지만, 네트워크 왕복과 검색 실행을 후보 수만큼 반복한다
- 캐시만 추가하면 이미 존재하는 동일 요청은 줄일 수 있지만, 처음 들어온 서로 다른 후보들의 검색 팬아웃은 해결하지 못한다
- 후보를 하나의 쿼리로 합치면 후보별 유사도 결과가 섞일 수 있어, 검색 단위는 유지하면서 전송만 묶어야 했다

대표 태그 결정에서 검색·응답까지

정확한 일치부터 유사도 판단까지 순서대로 확인한 뒤, 검색 요청을 묶어 RAG 응답으로 연결

STEP 4

TagService · 대표 태그 결정

일치 여부를 우선순위로 평가

STEP 4-1 · 우선순위 1

이름 완전 일치

기존 대표 태그 재사용

STEP 4-2 · 우선순위 2

벡터 유사도 ≥ 0.8

기존 대표 태그로 수렴

STEP 4-3 · 마지막

둘 다 없음

새 대표 태그 생성

STEP 5

검색 최적화 · _msearch

N개 KNN 쿼리를 1회 요청으로 묶음

STEP 6 · LLM

RAG 질의응답

검색 결과를 근거로 답변 생성

ACTION · 나의 판단과 행동

병목을 검색 계산과 요청 왕복으로 나눠 추적

순차 처리와 배치 처리의 검색 구간을 따로 측정해, 태그 추출기가 아니라 후보별 KNN 요청과 네트워크 왕복이 지연을 만든다는 점을 확인했다.

후보별 KNN 쿼리는 유지하고 _msearch로 전송만 묶음

각 후보가 독립된 유사도 결과를 받아야 하는 검색 의미는 바꾸지 않고, Elasticsearch _msearch에 N개의 KNN 쿼리를 한 요청으로 실어 보냈다.

배치 결과를 후보 순서대로 다시 연결

응답 배열을 원래 후보 순서에 맞춰 복원해, 이후의 완전 일치·벡터 유사도 판단이 개별 요청 방식과 같은 입력을 받도록 구성했다.

RESULT · 결과

키워드 후보 5개 기준 측정에서 Elasticsearch 요청은 5회에서 1회로, 해당 검색 구간은 250ms에서 10ms로 줄었다. 후보별 검색 의미를 유지한 채 통신 대기와 반복 실행을 함께 줄여, 문서 업로드 뒤 태그 정제 단계가 병목이 되지 않게 했다.

2025.08 ~ 2025.10

팀 프로젝트

장려상

맡은 역할

실시간 채팅 도메인

WEBSOCKET/STOMP 세션 인증

유기견 도메인

입양 신청 도메인

즐거찾기 도메인

Java 17

Spring Boot

Spring Security

JPA

PostgreSQL

Redis

OpenSearch

WebSocket/STOMP

Flyway

Docker Compose

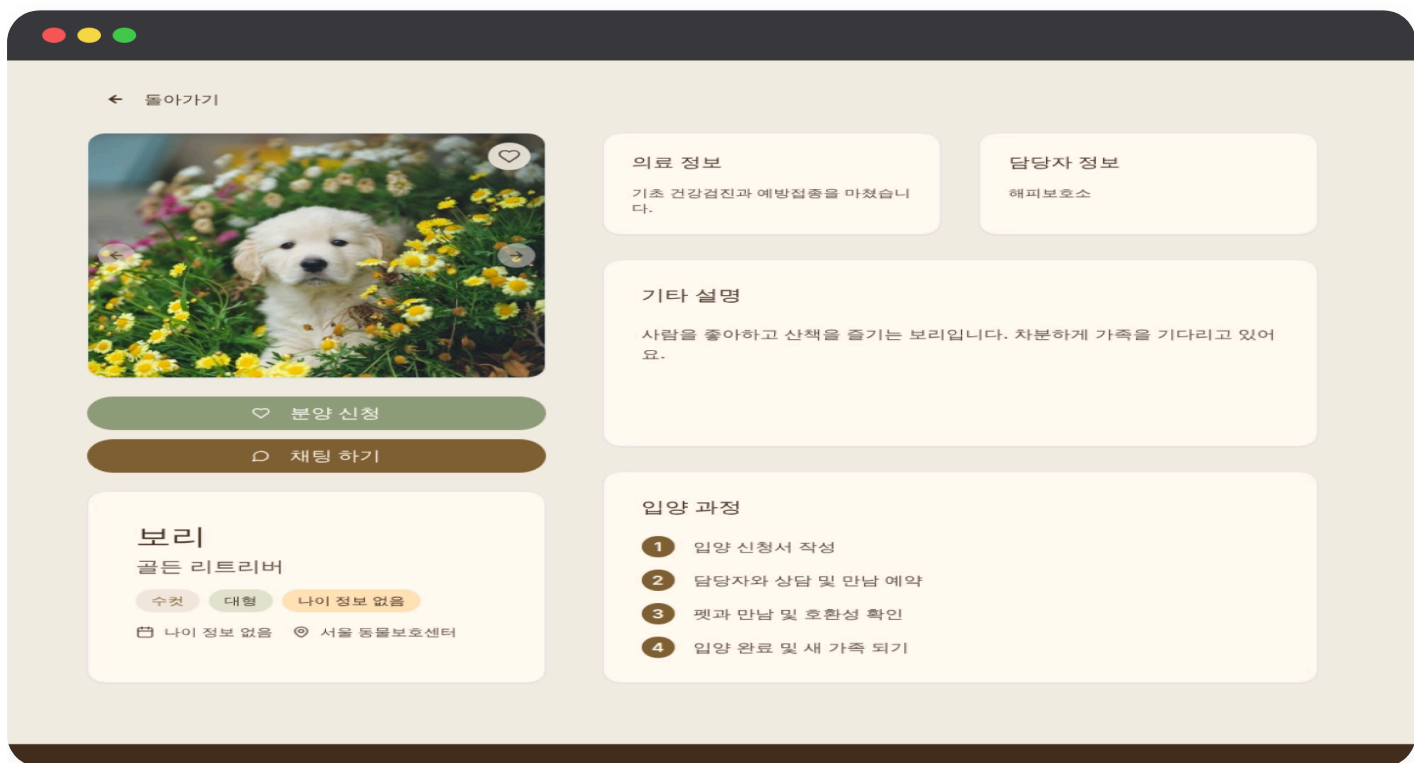
GitHub: github.com/orgs/Dangdaengdan/repositories · 담당 PR: github.com/search?q=org%3ADangdaengdan+author%3AChaehyunli+is%3Apr&type=pullrequests

PETNER

유기견 탐색·입양 신청·커뮤니티·보호소 실시간 채팅을 연결한 팀 백엔드 서비스

배경

왜 만들었나



유기견 상세에서 입양 신청과 보호소 채팅으로 이어지는 화면

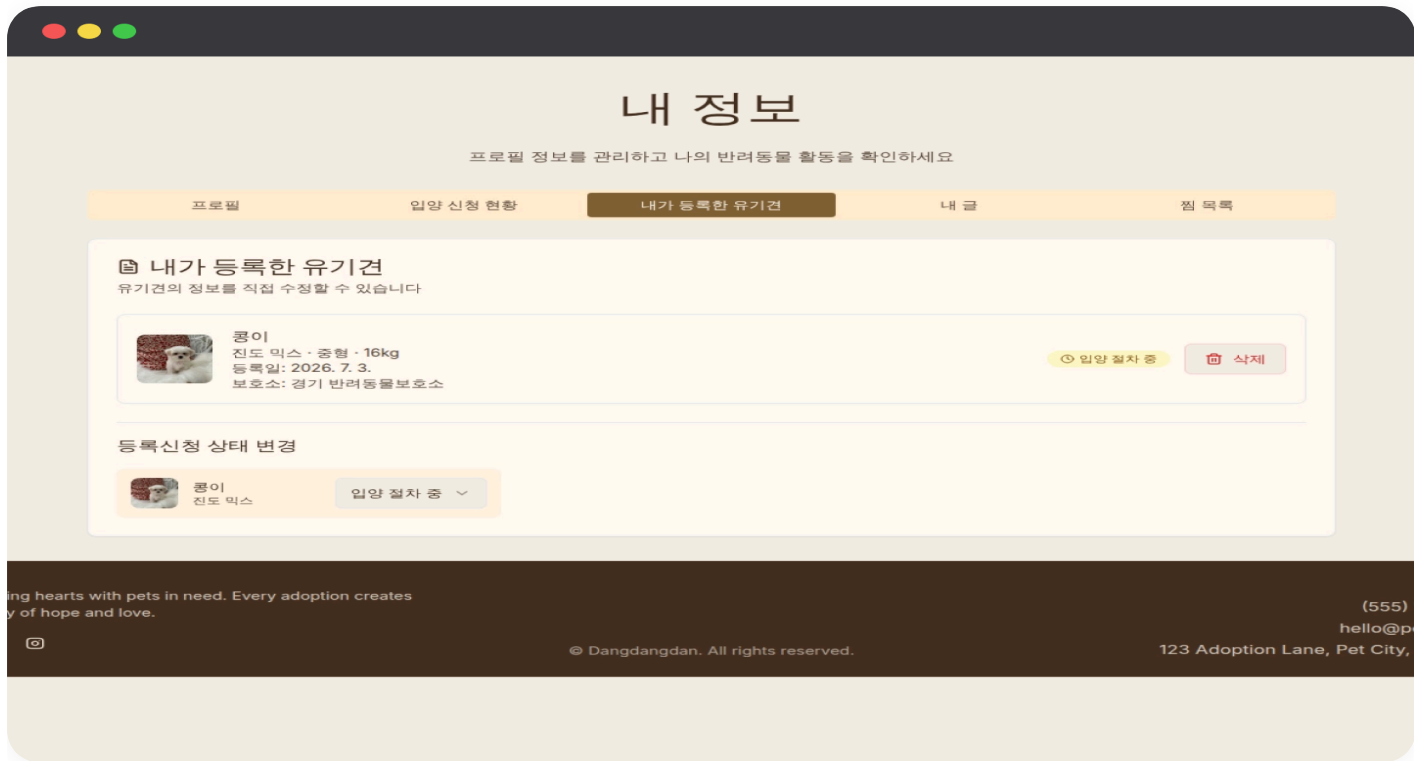
탐색 뒤에 끊기는 입양 경험

유기견 정보를 찾은 뒤 보호소에 문의하고 입양 절차를 밟는 흐름을 한 서비스 안에서 이어야 했다.

실시간 문의의 신뢰 경계

HTTP 로그인 상태를 WebSocket과 STOMP 메시지 처리까지 안전하게 이어야 했다.

탐색 이후의 입양 흐름을 백엔드로 연결했다



보호소가 등록한 유기견의 입양 절차 상태를 확인·변경하는 화면

유기견 정보와 입양 신청을 연결

상세 화면에서 선택한 유기견을 기준으로 신청이 생성되고, 보호소가 해당 요청을 이어서 처리할 수 있게 도메인을 구성했다.

보호소 기준의 상태 관리

보호소가 자신이 등록한 유기견과 현재 입양 절차 상태를 한 화면에서 확인하고 변경할 수 있도록 API를 구현했다.

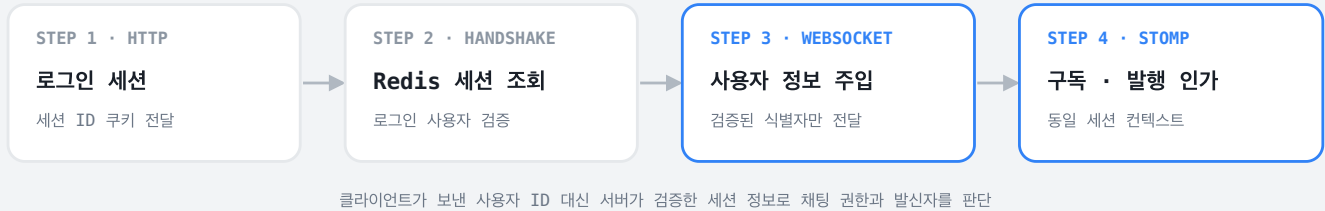
문제가 필요한 순간에는 채팅으로 전환

유기견 상세에서 보호소와의 실시간 상담으로 진입하도록 입양 신청 흐름과 채팅 도메인을 연결했다.

REST 요청은 HTTP 세션으로 로그인 사용자를 식별하지만, WebSocket은 연결 이후 STOMP 메시지를 별도로 처리한다. 연결만 허용하고 메시지 단계의 사용자 식별을 놓치면 채팅방 권한과 발신자를 신뢰할 수 없었다.

- 연결 직후의 인증 정보만 믿으면 이후 구독·발행 단계에서 동일 사용자인지 보장할 수 없음
- 메시지 본문에 사용자 식별자를 맡기면 클라이언트 값 위변조를 막을 수 없음

HTTP 세션을 STOMP 메시지까지 이어가는 인증 경계



ACTION · 나의 판단과 행동

핸드셰이크에서 Redis 세션을 직접 조회

HTTP 세션에 있는 로그인 사용자 정보를 확인한 뒤, 검증된 식별자만 WebSocket 세션 속성에 주입했다.

STOMP 처리 전 구간에 같은 컨텍스트를 사용

연결·구독·발행이 모두 세션 속성의 사용자 정보를 기준으로 동작하게 하고, 단일 HTML 테스트 페이지로 각 단계를 재현했다.

RESULT · 결과

HTTP 로그인 사용자와 STOMP 메시지 발신자를 같은 세션 컨텍스트로 연결했다. 채팅방의 권한 확인과 발신자 식별을 클라이언트 입력이 아닌 서버가 검증한 세션 정보에 맡겼다.

판단 02

채팅방에서 나가도 대화 이력까지 사라지면 안 됐다

SITUATION & TASK · 마주한 문제와 고민

채팅방 나가기를 Hard Delete로 처리하면 메시지와 FK 제약이 깨지고, 다시 입장했을 때 이전 입장 상담 이력을 이어 볼 수 없었다. 화면에서 사라지는 것과 데이터를 없애는 일은 다른 문제였다.

- 참여 레코드를 물리 삭제하면 연관 메시지의 무결성과 상담 이력을 함께 잃음
- 채팅방 목록에서 숨겨도 재입장 시 과거 메시지를 조회할 수 있어야 함

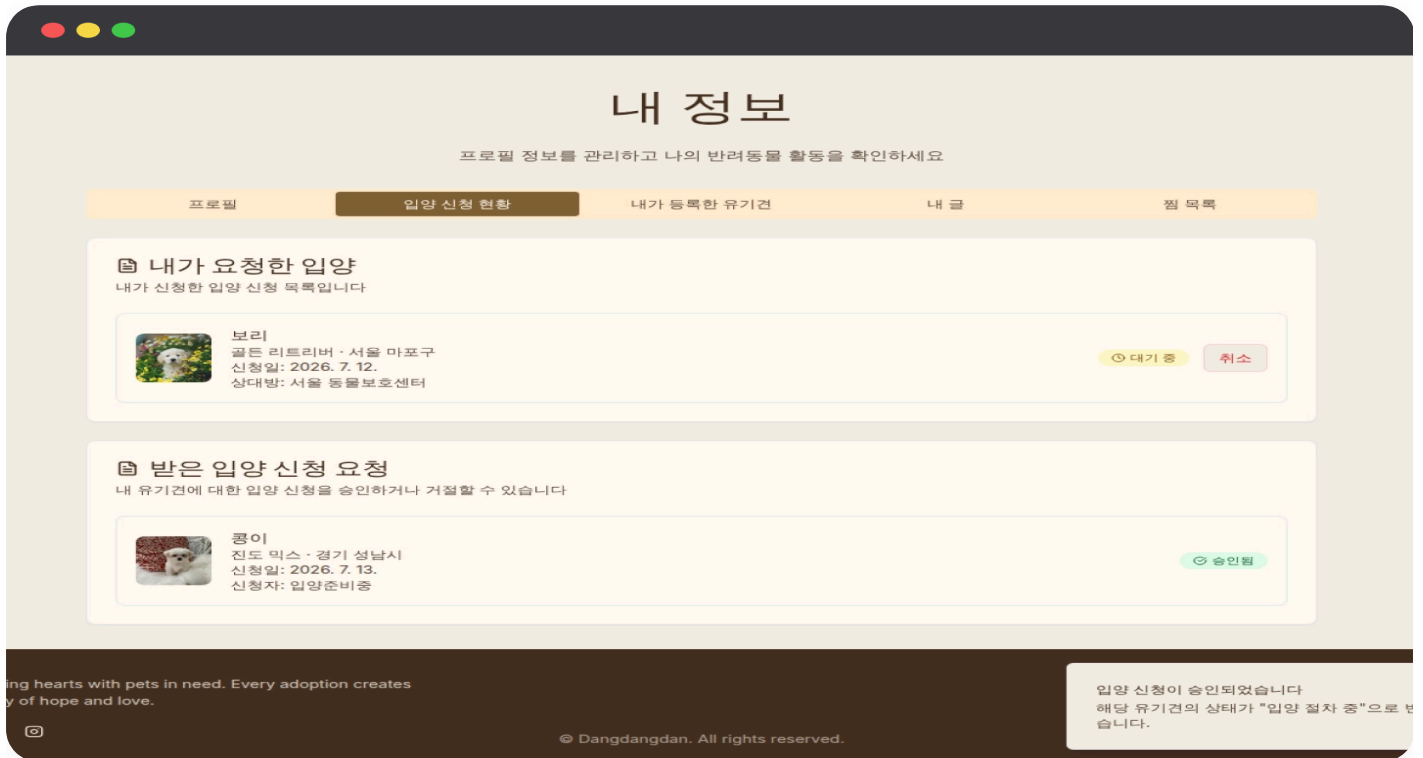
ACTION · 나의 판단과 행동

삭제를 참여 상태의 비활성화로 재정의

채팅방을 나간 사용자의 참여 레코드를 지우지 않고 Soft Delete 상태로 전환해, 메시지 관계는 그대로 보존했다.

목록과 이력을 서로 다른 기준으로 조회

채팅방 목록은 활성 참여자만 보이도록 필터링하고, 재입장 시에는 보존된 메시지를 페이지징 조회하도록 분리했다.



진행 상태와 함께 유지되는 사용자·보호소의 입양 신청 내역

RESULT · 결과

사용자는 채팅방을 떠난 것처럼 보이지만, 입양 상담 이력과 FK 무결성은 유지된다. 재입장 뒤에도 이전 메시지를 자연스럽게 이어 볼 수 있는 구조가 됐다.

2025.01 ~ 2025.03

팀 프로젝트

완성

맡은 역할

핵심 도메인 설계

REDIS 세션 인증 선택

리소스 단위 RBAC

지원·승인 흐름 구현

Java

Spring Boot

Spring Security

JPA

MySQL

Redis

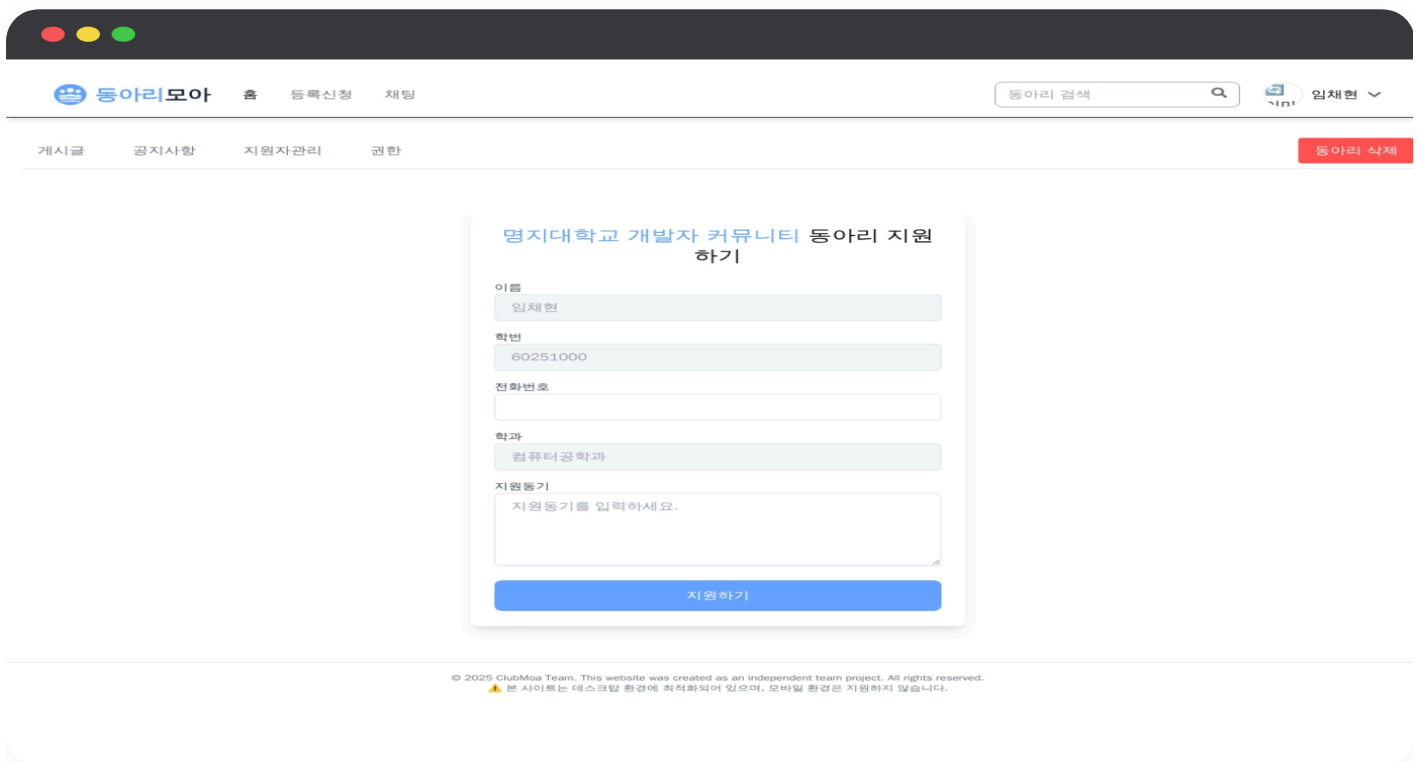
GitHub: github.com/Chaehyunli/TeamProject2025

동아리모아

동아리 탐색·지원·운영·권한 위임을 한 서비스에서 다룬 팀 백엔드 프로젝트

배경

왜 만들었나



동아리 탐색 이후 지원서를 작성하는 사용자 흐름

탐색과 운영이 분리된 동아리 활동

사용자는 동아리를 찾고 지원하며, 운영자는 지원자를 승인하고 구성원을 관리해야 했다.

동아리마다 달라지는 사용자 역할

같은 사용자도 한 동아리에서는 회장, 다른 동아리에서는 일반 회원일 수 있었다.

즉시 통제가 필요한 운영 기능

강제 탈퇴·계정 정지·권한 변경이 발생하면 기존 접근 권한도 바로 차단돼야 했다.

판단 01

토큰의 편의성보다 운영자가 즉시 통제할 수 있는 인증이 필요했다

SITUATION & TASK · 마주한 문제와 고민

동아리 강제 탈퇴나 계정 정지 이후에도 JWT는 만료 시점까지 유효하다. 운영자가 접근을 차단해도 이미 발급된 토큰으로 요청을 계속 보낼 수 있어, 즉시 통제가 필요한 운영 서비스와 맞지 않았다.

- JWT 만료 시간을 짧게 잡아도 정지 시점부터 만료까지 접근 가능한 공백은 남음
- 토큰 블랙리스트를 별도로 관리하면 결국 서버 상태가 필요해 JWT의 단순성이 줄어들

ACTION · 나의 판단과 행동

인증 상태를 서버에서 통제

로그인 상태를 Redis 세션으로 관리해 서버가 사용자의 세션 유효성을 직접 판단하도록 설계했다.

운영 이벤트와 세션 무효화를 연결

로그아웃뿐 아니라 계정 정지·강제 탈퇴 시에도 세션을 제거해 이후 요청을 즉시 차단할 수 있게 했다.

RESULT · 결과

계정 상태가 바뀌면 서버가 Redis 세션을 즉시 무효화할 수 있어, 이미 로그인한 사용자에게도 운영 정책을 바로 적용하는 인증 구조를 확보했다.

판단 02

권한은 사용자 전역이 아니라 동아리마다 달라져야 했다

SITUATION & TASK · 마주한 문제와 고민

단순한 ADMIN·USER 전역 역할만으로는 한 사용자가 동아리마다 다른 역할을 갖는 구조를 표현할 수 없었다. 특정 동아리의 회장만 권한을 위임하고 구성원을 관리하게 하려면 권한의 범위를 해당 동아리 리소스로 좁혀야 했다.

- 사용자 계정에 역할을 저장하면 한 사용자가 여러 동아리에서 서로 다른 역할을 갖는 상황을 표현할 수 없음
- 화면에서 버튼만 숨기는 방식으로는 API 직접 호출을 막을 수 없어 서버에서 리소스 기준 인가가 필요함

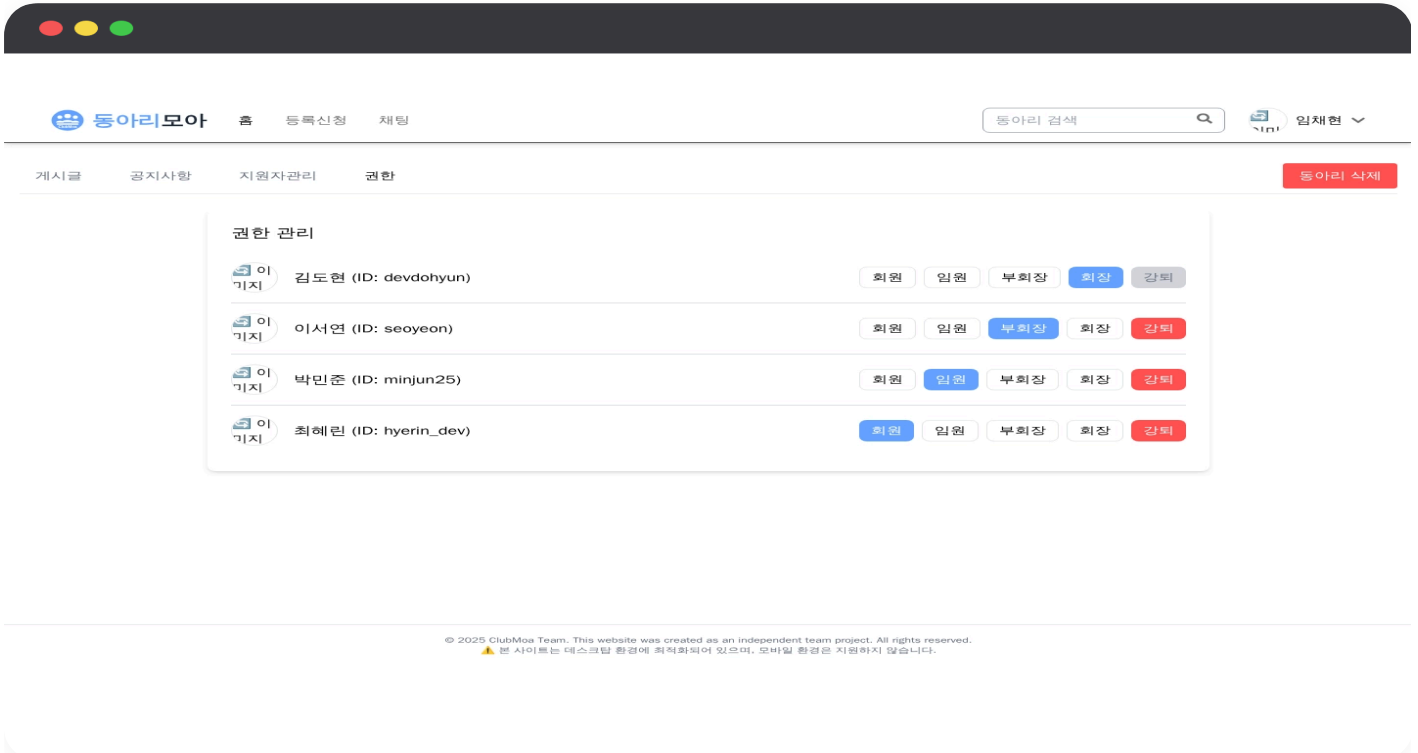
ACTION · 나의 판단과 행동

사용자·동아리·역할을 하나의 관계로 설계

사용자에게 역할을 직접 부여하지 않고, 사용자와 동아리의 관계에 역할을 두어 같은 사용자가 동아리마다 다른 권한을 갖게 했다.

요청 대상 동아리를 기준으로 권한 확인

권한 위임과 구성원 관리는 요청한 사용자가 해당 동아리에서 가진 역할을 확인한 뒤 실행되도록 인가 범위를 제한했다.



동아리별 역할 위임과 구성원 강퇴를 처리하는 권한 관리 화면

RESULT · 결과

동일한 사용자가 동아리마다 독립된 역할을 가질 수 있고, 권한 위임과 구성원 강퇴가 요청 대상 동아리의 권한 범위 안에서만 실행되도록 만들었다.

동아리 탐색 → 지원 → 승인 → 역할 위임 흐름 구현 · 2025.01 ~ 2025.03 완성

2025.03 ~ 2025.10

팀 프로젝트

완성

맡은 역할

백엔드 도메인 ERD

XGBOOST 감정 예측 모델

합성 데이터 생성 규칙

PYGAME → REACT 전환

Python

FastAPI

XGBoost

React

PostgreSQL

Docker

GitHub: github.com/orgs/no-so-gong/repositories · 담당 PR: github.com/search?q=org%3Ano-so-gong+author%3AChaehyunli+is%3Apr&type=pullrequests

🎮 노소공

행동 데이터로 펫 감정을 예측하고 미니게임 보상·성장 흐름을 연결한 ML 기반 동물 육성 게임

배경

왜 만들었나



감정·친밀도·재화와 상호작용을 한 화면에 담은 펫 육성 게임

행동이 상태를 바꾸는 육성 게임

놀이·밥 주기·선물하기 같은 행동을 펫의 감정과 성장 상태에 연결해야 했다.

서비스 초기의 데이터 부족

실제 사용자 행동 로그가 없어 감정 예측 모델을 바로 학습할 수 없었다.

예측을 실제 상호작용에 연결

모델 결과를 화면에 표시하는 데서 끝내지 않고 활동·보상·성장 흐름에 반영했다.

판단 01

실사용 로그가 없다고 감정 예측을 포기하는 대신, 행동 규칙을 먼저 데이터로 만들었다

SITUATION & TASK · 마주한 문제와 고민

서비스 초기에는 지도학습에 사용할 실제 사용자 행동 로그가 없었다. 임의의 난수로 데이터를 만들면 행동과 감정 사이의 관계를 학습할 수 없기 때문에, 게임에서 의도한 상태 변화 규칙을 데이터 생성 기준으로 먼저 정의해야 했다.

- 실사용 로그가 쌓일 때까지 기다리면 감정 예측과 후속 게임 흐름을 함께 검증할 수 없음
- 무작위 합성 데이터는 행동과 감정의 관계를 보존하지 못하고, 합성 데이터 성능을 실제 사용자 성능으로 해석해서도 안 됨

ACTION · 나의 판단과 행동

행동과 감정 사이의 규칙을 먼저 정의

사용자의 행동과 펫 상태가 감정 값에 어떤 방향으로 영향을 주는지 정리하고, 이 규칙을 바탕으로 합성 데이터 10,000건을 생성했다.

비선형 관계를 학습할 수 있는 XGBoost 선택

행동·상태 조합에 따라 달라지는 감정 값을 학습하도록 XGBoost 회귀 모델을 구성하고 테스트셋에서 평가했다.

RESULT · 결과

합성 데이터 테스트셋에서 R^2 0.9964, RMSE 0.22를 확인했다. 이 결과는 실제 사용자에게 대한 일반화 성능이 아니라, 정의한 행동 규칙을 모델이 재현할 수 있는지 검증한 수치로 한정했다.



감정 예측의 입력이 되는 산책·공놀이·애견카페 활동 선택

판단 02

Pygame 코드를 그대로 옮기지 않고 게임 동작의 설계도로 사용했다

SITUATION & TASK · 마주한 문제와 고민

초기 미니게임은 Pygame으로 구현했지만 브라우저 기반 서비스와 직접 통합할 수 없었다. 화면 코드를 그대로 번역하면 게임 로직과 UI 상태가 한 컴포넌트에 뒤섞일 가능성이 컸다.

- Pygame 실행 환경을 그대로 유지하면 React 기반 웹 서비스 안에서 같은 사용자 흐름으로 제공할 수 없음
- 프레임 단위 화면 코드를 그대로 번역하면 렌더링과 게임 규칙이 결합돼 상태 변경을 추적하기 어려움

ACTION · 나의 판단과 행동

기존 구현에서 게임 규칙을 먼저 분리

점수 계산, 상태 변화, 입력 이벤트를 Pygame 화면 코드에서 분리해 웹 버전의 동작 기준으로 사용했다.

상태와 이벤트를 React 구조로 재구현

게임 상태를 React에서 관리하고 사용자 입력과 화면 갱신을 컴포넌트 흐름에 맞게 다시 구성했다.

RESULT · 결과

Pygame에서 검증한 게임 규칙을 유지하면서 입력·상태·화면 갱신을 React 흐름으로 전환해, 미니게임을 브라우저 기반 육성 서비스 안에 통합했다.

Cold Start를 규칙 기반 합성 데이터로 풀어 감정 예측 모델과 후속 게임 흐름을 함께 검증했다. 정의한 행동 규칙을 모델이 재현하는지 본 수치는 합성 테스트셋 R^2 0.9964 / RMSE 0.22이며, 실사용 일반화 성능은 아니다. 2025.03 ~ 2025.10 완성